Day05 出行驻留识别（阶段2）：k-means聚类与最优K选择

> 课程：交通时空大数据分析与可视化系统（Python 版） | 第 5 天（8 课时）
> 主题：深入理解 k-means 聚类算法原理，手写实现 k-means，掌握聚类评估指标与最优K选择

一、本日目标

1. 理解 k-means 聚类算法原理（目标函数、迭代过程、收敛条件）；
2. 手写实现 k-means（不调用 sklearn，理解每一步）；
3. 掌握距离矩阵计算与初始化策略（随机、k-means++）；
4. 掌握聚类评估指标（SSE、轮廓系数、邓恩指数）；
5. 掌握最优K选择方法（K=2~10遍历，多指标综合判断）；
6. 掌握单点成簇判定（单点成簇=出行点）；
7. 完整实现阶段2模块（tr_stage2.py）；
8. 分析阶段2结果并可视化（聚类结果、最优K分布）。

二、知识要点

- k-means 是最经典的划分式聚类算法，目标是最小化簇内平方和（SSE）；
- 本项目对每个用户独立做 k-means，K 从 2 到 10 遍历，用邓恩指数选最优K；
- 关键坑：K 上限不能超过用户点数（否则报 "Cannot take a larger sample than population"）；
- 单点成簇（簇内只有1个点）标记为出行点，进入阶段3；
- 本阶段输出：stage2_cluster.csv（聚类结果）、stage2_travel_points.csv（单点成簇的出行点）。

课时1：k-means算法原理

1.1 课时目标

深入理解 k-means 聚类的目标函数、迭代过程、收敛条件和优缺点。

1.2 知识点深度讲解

1.2.1 什么是聚类

聚类（Clustering）是无监督学习的一种，目标是把相似的样本分到同一组（簇），不相似的样本分到不同组。

与分类的区别：
- 分类（Classification）：有标签，监督学习，预测类别；
- 聚类（Clustering）：无标签，无监督学习，发现内在结构。

本项目中的聚类目标：
对每个用户的位置点（经纬度）聚类，发现用户的"驻留区域"（如家、公司、常去的商场）。同一个驻留区域内的点经纬度接近，应该被聚到同一个簇。

1.2.2 k-means 的目标函数

k-means 的目标是最小化**簇内平方和**（Within-Cluster Sum of Squares, WCSS / SSE）：

$$
SSE = \sum_{i=1}^{k} \sum_{x \in C_i} |x - \mu_i|^2
$$

其中：
- k：簇的数量；
- C_i：第 i 个簇；
- μ_i：第 i 个簇的中心（均值）；
- x：样本点；
- $|x - \mu_i|^2$：样本点到簇中心的距离平方。

直观理解：SSE 衡量的是"所有点到各自簇中心的距离平方和"。SSE 越小，说明簇内点越紧密，聚类效果越好。

1.2.3 k-means 的迭代过程

k-means 是一个迭代算法，分为两步交替进行：

步骤1：分配（Assignment）
把每个样本点分配给距离最近的簇中心：

$$
C_i = {x : |x - \mu_i| \le |x - \mu_j|, \forall j}
$$

步骤2：更新（Update）
重新计算每个簇的中心（簇内所有点的均值）：

$$
\mu_i = \frac{1}{|C_i|} \sum_{x \in C_i} x
$$

完整流程：

```
输入：样本集 X，簇数 k
输出：簇划分 C，簇中心 μ

1. 初始化：随机选择 k 个点作为初始簇中心 μ₁, μ₂, ..., μₖ
2. 重复：
   a. 分配：把每个点分配给距离最近的簇中心
   b. 更新：重新计算每个簇的中心（簇内点的均值）
   c. 检查收敛：如果簇中心不再变化（或变化很小），停止
3. 返回簇划分和簇中心
```

1.2.4 收敛条件

k-means 的收敛条件通常有以下几种：
1. **簇中心不再变化**：新旧簇中心完全相同（或距离小于阈值）；
2. **簇划分不再变化**：所有点的簇标签不再变化；
3. **SSE 不再下降**：SSE 的变化量小于阈值；
4. **达到最大迭代次数**：防止无限循环（通常设为 100-300 次）。

> k-means 一定会收敛，因为 SSE 在每次迭代后单调不增，且有下界（0）。但可能收敛到局部最优，不一定是全局最优。

1.2.5 k-means 的优缺点

优点：
1. 简单易懂，实现容易；
2. 计算效率高，时间复杂度 O(nkt)（n=样本数，k=簇数，t=迭代次数）；
3. 对大数据集可扩展；
4. 结果可解释（簇中心有实际意义）。

缺点：
1. 需要预先指定 k（簇数）；
2. 对初始簇中心敏感，可能收敛到局部最优；
3. 对异常值敏感（异常值会拉偏簇中心）；
4. 只能发现球形簇（假设簇是凸的、各向同性的）；
5. 不适合处理非凸形状的簇（如月牙形、环形）。

1.2.6 本项目中的 k-means 应用

本项目对每个用户独立做 k-means：
1. **输入**：该用户的非阶段1出行点（经纬度）；
2. **K 范围**：2 到 10（用户可能有多个驻留区域，如家、公司、商场）；
3. **最优K选择**：遍历 K=2~10，用邓恩指数选最优；
4. **输出**：每个点的簇标签、簇中心；
5. **单点成簇**：如果某个簇只有1个点，标记为出行点（不像是驻留区域）。

> 为什么对每个用户独立聚类？因为不同用户的驻留区域不同，混在一起聚类没有意义。每个用户的驻留区域数量也不同（有人只有家和公司，有人有多个常去地点）。

1.3 项目实现步骤

本课时不写代码，通过公式和流程图理解 k-means 原理。

1.4 完整代码（k-means 原理演示）

```
# ============================================================
# 课时1：k-means算法原理演示
# 功能：用简单数据演示 k-means 的迭代过程
# ============================================================

import numpy as np
import matplotlib.pyplot as plt

plt.rcParams["font.sans-serif"] = ["SimHei"]
plt.rcParams["axes.unicode_minus"] = False

# 生成3个簇的模拟数据
np.random.seed(42)
cluster1 = np.random.randn(50, 2) + np.array([0, 0])       # 中心(0,0)
cluster2 = np.random.randn(50, 2) + np.array([5, 5])       # 中心(5,5)
cluster3 = np.random.randn(50, 2) + np.array([0, 5])       # 中心(0,5)
X = np.vstack([cluster1, cluster2, cluster3])

print(f"数据：{X.shape[0]} 个点，{X.shape[1]} 维")
print(f"真实簇中心：(0,0)，(5,5)，(0,5)")

# k-means 参数
k = 3
max_iter = 100
tol = 1e-4   # 收敛阈值

# 步骤1：初始化（随机选择k个点作为初始中心）
np.random.seed(0)
initial_indices = np.random.choice(len(X), k, replace=False)
centers = X[initial_indices].copy()
print(f"\n初始簇中心:\n{centers}")

# 迭代
for iteration in range(max_iter):
    # 步骤2a：分配（每个点到最近的中心）
    # 计算距离矩阵：(n, k)
```

```
    # X[:, None, :] → (n, 1, 2)
    # centers[None, :, :] → (1, k, 2)
    # 广播后 → (n, k, 2)
    diff = X[:, None, :] - centers[None, :, :]
    dist_sq = (diff ** 2).sum(axis=2)   # (n, k)
    labels = dist_sq.argmin(axis=1)      # 每个点最近的簇

    # 步骤2b：更新（重新计算簇中心）
    new_centers = np.zeros_like(centers)
    for i in range(k):
        cluster_points = X[labels == i]
        if len(cluster_points) > 0:
            new_centers[i] = cluster_points.mean(axis=0)
        else:
            new_centers[i] = centers[i]   # 空簇保持原中心

    # 步骤2c：检查收敛
    center_shift = np.linalg.norm(new_centers - centers, axis=1).max()
    sse = dist_sq.min(axis=1).sum()

    if iteration < 5 or iteration % 10 == 0:
        print(f"\n迭代 {iteration+1}:")
        print(f"  簇中心:\n{new_centers}")
        print(f"  中心最大移动: {center_shift:.6f}")
        print(f"  SSE: {sse:.4f}")

    centers = new_centers

    if center_shift < tol:
        print(f"\n收敛！迭代 {iteration+1} 次后中心移动 < {tol}")
        break

# 最终结果
print(f"\n最终簇中心:\n{centers}")
print(f"最终 SSE: {sse:.4f}")
print(f"各簇点数: {[np.sum(labels == i) for i in range(k)]}")

# 可视化
```

```
fig, axes = plt.subplots(1, 2, figsize=(14, 6))

# 原始数据
axes[0].scatter(X[:, 0], X[:, 1], c="gray", s=10, alpha=0.5)
axes[0].set_title("原始数据（3个真实簇）")
axes[0].set_xlabel("X1")
axes[0].set_ylabel("X2")

# 聚类结果
colors = ["red", "blue", "green"]
for i in range(k):
    mask = labels == i
    axes[1].scatter(X[mask, 0], X[mask, 1], c=colors[i], s=10, alpha=0.5, label=f"簇{i}")
axes[1].scatter(centers[:, 0], centers[:, 1], c="black", marker="*", s=200, label="簇中心")
axes[1].set_title(f"k-means 聚类结果（k={k}, SSE={sse:.2f}）")
axes[1].set_xlabel("X1")
axes[1].set_ylabel("X2")
axes[1].legend()

plt.tight_layout()
plt.savefig("kmeans_demo.png", dpi=150, bbox_inches="tight")
plt.close()
print("\n可视化已保存：kmeans_demo.png")
```

1.5 运行结果与解读

运行后展示：
- 初始簇中心（随机选择）；
- 迭代过程（前5次+每10次打印）；
- 最终簇中心（接近真实中心 (0,0), (5,5), (0,5)）；
- SSE 逐步下降，最终收敛；
- 可视化对比原始数据和聚类结果。

> 关键理解：
> 1. k-means 通过"分配-更新"两步交替迭代，SSE 单调下降；

> 2. 最终簇中心接近真实中心，说明聚类成功；
> 3. SSE 衡量聚类质量，越小越好。

1.6 当堂练习

1. 修改初始随机种子（np.random.seed），观察是否每次都收敛到相同结果；
2. 把 k 改为 2 和 4，观察 SSE 和聚类结果的变化；
3. 思考：为什么 k-means 可能收敛到局部最优？（提示：初始中心的选择影响最终结果）

1.7 课时小结

- k-means 目标：最小化 SSE（簇内平方和）；
- 迭代过程：分配（点到最近中心）→ 更新（中心=簇内均值）→ 检查收敛；
- 收敛条件：中心不再变化、SSE 不再下降、达到最大迭代次数；
- 优点：简单、高效、可解释；缺点：需指定k、对初始值敏感、对异常值敏感；
- 本项目对每个用户独立聚类，K=2~10，邓恩指数选最优。

课时2：手写k-means实现

2.1 课时目标

不调用 sklearn，手写实现完整的 k-means 算法，理解每一行代码。

2.2 知识点深度讲解

2.2.1 实现框架

手写 k-means 的核心函数：

```
def kmeans(X, k, max_iter=300, tol=1e-4, random_state=None):
    """
    k-means 聚类

    Args:
        X: (n, d) 样本矩阵
        k: 簇数
        max_iter: 最大迭代次数
        tol: 收敛阈值
        random_state: 随机种子

    Returns:
        labels: (n,) 每个点的簇标签
        centers: (k, d) 簇中心
        sse: 最终SSE
        n_iter: 实际迭代次数
    """
    pass
```

2.2.2 初始化策略

随机初始化：从样本中随机选择 k 个点作为初始中心。

```
rng = np.random.default_rng(random_state)
indices = rng.choice(n, k, replace=False)
centers = X[indices].copy()
```

k-means++ 初始化（更优，后续课时详解）：
- 第一个中心随机选择；
- 后续中心以"距离已选中心越远，概率越大"的方式选择；
- 优点：初始中心分散，收敛更快，SSE 更低。

本课时先用随机初始化，k-means++ 在课时3详解。

2.2.3 分配步骤（向量化）

用广播计算距离矩阵：

```
# X: (n, d), centers: (k, d)
# X[:, None, :] → (n, 1, d)
# centers[None, :, :] → (1, k, d)
# diff → (n, k, d)
diff = X[:, None, :] - centers[None, :, :]
dist_sq = (diff ** 2).sum(axis=2)   # (n, k)
labels = dist_sq.argmin(axis=1)      # (n,)
```

2.2.4 更新步骤

```
new_centers = np.zeros_like(centers)
for i in range(k):
    mask = labels == i
    if mask.sum() > 0:
        new_centers[i] = X[mask].mean(axis=0)
    else:
        # 空簇：重新随机选择一个点
        new_centers[i] = X[rng.integers(n)]
```

> 空簇处理：如果某个簇没有点，需要重新初始化，否则该簇会消失。简单做法是随机选择一个点作为新中心。

2.2.5 收敛检查

```
center_shift = np.linalg.norm(new_centers - centers, axis=1).max()
if center_shift < tol:
    break
centers = new_centers
```

2.2.6 SSE 计算

```
sse = dist_sq.min(axis=1).sum()
```

2.3 项目实现步骤

1. 实现 kmeans 函数（初始化、分配、更新、收敛检查）；

2. 用模拟数据测试；

3. 对比 sklearn 的 KMeans 结果。

2.4 完整代码（手写k-means）

```
# ============================================================
# 课时2：手写k-means实现
# ============================================================

import numpy as np


def kmeans(X, k, max_iter=300, tol=1e-4, random_state=None):
    """
    手写 k-means 聚类算法。

    Args:
        X: ndarray, shape (n_samples, n_features)，样本矩阵
        k: int，簇的数量
        max_iter: int，最大迭代次数
        tol: float，收敛阈值（中心移动小于此值则收敛）
        random_state: int，随机种子

    Returns:
        labels: ndarray, shape (n_samples,)，每个样本的簇标签
        centers: ndarray, shape (k, n_features)，簇中心
        sse: float，最终的簇内平方和
        n_iter: int，实际迭代次数
    """
    n, d = X.shape
    rng = np.random.default_rng(random_state)

    # ------------------------------------------------------------
    # 步骤1：初始化（随机选择k个点作为初始中心）
    # ------------------------------------------------------------
    indices = rng.choice(n, k, replace=False)   # 不重复选择
    centers = X[indices].copy()

    # ------------------------------------------------------------
    # 步骤2：迭代
    # ------------------------------------------------------------
```

```
    for iteration in range(max_iter):
        # 步骤2a：分配（每个点到最近的中心）
        # 用广播计算距离矩阵 (n, k)
        # X[:, None, :] → (n, 1, d)
        # centers[None, :, :] → (1, k, d)
        # 广播后 diff → (n, k, d)
        diff = X[:, None, :] - centers[None, :, :]
        dist_sq = (diff ** 2).sum(axis=2)   # (n, k)，每个点到每个中心的距离平方
        labels = dist_sq.argmin(axis=1)      # (n,)，每个点最近的簇标签

        # 步骤2b：更新（重新计算每个簇的中心）
        new_centers = np.zeros_like(centers)
        for i in range(k):
            mask = labels == i
            if mask.sum() > 0:
                # 簇内点的均值作为新中心
                new_centers[i] = X[mask].mean(axis=0)
            else:
                # 空簇：随机选择一个点作为新中心
                new_centers[i] = X[rng.integers(n)]

        # 步骤2c：检查收敛
        center_shift = np.linalg.norm(new_centers - centers, axis=1).max()
        centers = new_centers

        if center_shift < tol:
            break

    # ------------------------------------------------------------
    # 步骤3：计算最终 SSE
    # ------------------------------------------------------------
    diff = X[:, None, :] - centers[None, :, :]
    dist_sq = (diff ** 2).sum(axis=2)
    labels = dist_sq.argmin(axis=1)
    sse = dist_sq.min(axis=1).sum()

    return labels, centers, sse, iteration + 1
```

```
# ============================================================
# 测试手写 k-means
# ============================================================
if __name__ == "__main__":
    # 生成3个簇的模拟数据
    np.random.seed(42)
    cluster1 = np.random.randn(100, 2) + np.array([0, 0])
    cluster2 = np.random.randn(100, 2) + np.array([5, 5])
    cluster3 = np.random.randn(100, 2) + np.array([0, 5])
    X = np.vstack([cluster1, cluster2, cluster3])

    print(f"测试数据: {X.shape[0]} 个点, {X.shape[1]} 维")
    print(f"真实簇中心: (0,0), (5,5), (0,5)")

    # 运行手写 k-means
    labels, centers, sse, n_iter = kmeans(X, k=3, random_state=42)

    print(f"\n手写 k-means 结果:")
    print(f"  迭代次数: {n_iter}")
    print(f"  SSE: {sse:.4f}")
    print(f"  簇中心:\n{centers}")
    print(f"  各簇点数: {[np.sum(labels == i) for i in range(3)]}")

    # 对比 sklearn（如果安装了）
    try:
        from sklearn.cluster import KMeans
        km = KMeans(n_clusters=3, random_state=42, n_init=10)
        km.fit(X)
        print(f"\nsklearn KMeans 结果:")
        print(f"  迭代次数: {km.n_iter_}")
        print(f"  SSE: {km.inertia_:.4f}")
        print(f"  簇中心:\n{km.cluster_centers_}")
        print(f"  各簇点数: {[np.sum(km.labels_ == i) for i in range(3)]}")

        print(f"\nSSE 差异: {abs(sse - km.inertia_):.4f}（越小说明手写实现越准确）")
    except ImportError:
        print("\n（未安装 sklearn，跳过对比）")
```

```
# 多次运行观察稳定性
print("\n多次运行观察稳定性（不同随机种子）:")
for seed in range(5):
    labels, centers, sse, n_iter = kmeans(X, k=3, random_state=seed)
    print(f"  seed={seed}: SSE={sse:.4f}, 迭代={n_iter}")
```

2.5 运行结果与解读

运行后展示：
- 手写 k-means 的结果（迭代次数、SSE、簇中心、各簇点数）；
- 与 sklearn KMeans 的对比（SSE 差异很小，说明手写实现正确）；
- 多次运行观察稳定性（不同随机种子可能得到不同 SSE，说明对初始值敏感）。

> 关键理解：
> 1. 手写 k-means 的核心是"分配-更新"两步迭代；
> 2. 距离矩阵用广播实现，向量化计算，速度快；
> 3. 空簇需要重新初始化，否则簇会消失；
> 4. 对初始值敏感，多次运行可能得到不同结果。

2.6 当堂练习

1. 修改空簇处理策略（改为选择距离当前中心最远的点），观察效果；
2. 用真实的经纬度数据（清洗后数据的某个用户）测试手写 k-means；
3. 思考：为什么 k-means 对初始值敏感？有什么解决方法？（提示：多次运行取最优、k-means++初始化）

2.7 课时小结

- 手写 k-means：初始化→分配（广播距离矩阵）→更新（簇内均值）→收敛检查；
- 空簇处理：随机选择新中心；
- 距离矩阵用广播实现，向量化计算；
- 对初始值敏感，多次运行可能不同；
- 与 sklearn 结果对比，SSE 差异很小，说明实现正确。

课时3：距离矩阵与初始化策略

3.1 课时目标

深入理解距离矩阵的计算方法，掌握随机初始化和 k-means++ 初始化策略。

3.2 知识点深度讲解

3.2.1 距离矩阵的计算

距离矩阵是 k-means 分配步骤的核心，形状为 (n, k)，第 i 行第 j 列表示第 i 个点到第 j 个簇中心的距离平方。

方法1：双重循环（慢）

```
dist_sq = np.zeros((n, k))
for i in range(n):
    for j in range(k):
        dist_sq[i, j] = np.sum((X[i] - centers[j]) ** 2)
```

方法2：单循环（中）

```
dist_sq = np.zeros((n, k))
for j in range(k):
    dist_sq[:, j] = np.sum((X - centers[j]) ** 2, axis=1)
```

方法3：广播（快，推荐）

```
# X: (n, d), centers: (k, d)
diff = X[:, None, :] - centers[None, :, :]   # (n, k, d)
dist_sq = (diff ** 2).sum(axis=2)              # (n, k)
```

方法4：矩阵展开（最快）

利用公式 $|x - c|^2 = |x|^2 + |c|^2 - 2x \cdot c$：

```
X_sq = np.sum(X ** 2, axis=1, keepdims=True)   # (n, 1)
C_sq = np.sum(centers ** 2, axis=1, keepdims=True).T  # (1, k)
cross = X @ centers.T   # (n, k)
dist_sq = X_sq + C_sq - 2 * cross   # (n, k)
```

> 方法4利用矩阵乘法，速度最快，尤其当 n 和 k 较大时。但方法3更直观，本项目用方法3即可（数据量不大）。

3.2.2 随机初始化

随机初始化是最简单的初始化策略：从样本中随机选择 k 个不重复的点作为初始中心。

```
rng = np.random.default_rng(random_state)
indices = rng.choice(n, k, replace=False)
centers = X[indices].copy()
```

优点：简单、快速；
缺点：可能选到相近的点，导致初始中心不分散，收敛慢，SSE 高。

3.2.3 k-means++ 初始化

k-means++ 是一种更优的初始化策略，核心思想是"初始中心应该尽可能分散"。

算法步骤：
1. 第一个中心：从样本中随机选择一个点；
2. 第 i 个中心（i=2,3,...,k）：
a. 计算每个点到已选中心的最短距离平方 $d(x)^2$；
b. 以概率 $p(x) = \frac{d(x)^2}{\sum d(x)^2}$ 选择下一个中心；
c. 距离越远的点被选中的概率越大。

为什么 k-means++ 更好?
- 初始中心分散，避免选到相近的点；
- 收敛更快（迭代次数更少）；
- SSE 更低（聚类质量更好）；
- 理论证明：k-means++ 的期望 SSE 是最优解的 O(log k) 倍。

k-means++ 实现：

```
def kmeans_plus_plus_init(X, k, random_state=None):
    """k-means++ 初始化。"""
    n, d = X.shape
    rng = np.random.default_rng(random_state)
    centers = []

    # 第一个中心：随机选择
    first_idx = rng.integers(n)
    centers.append(X[first_idx])

    # 后续中心：按距离概率选择
    for _ in range(1, k):
        # 计算每个点到已选中心的最短距离平方
        centers_arr = np.array(centers)
        diff = X[:, None, :] - centers_arr[None, :, :]
        dist_sq = (diff ** 2).sum(axis=2)   # (n, len(centers))
        min_dist_sq = dist_sq.min(axis=1)    # (n,)

        # 按概率选择
        probs = min_dist_sq / min_dist_sq.sum()
        next_idx = rng.choice(n, p=probs)
        centers.append(X[next_idx])

    return np.array(centers)
```

3.2.4 本项目中的初始化

本项目用随机初始化（简单，且每个用户数据量不大，多次运行取最优即可）。但可以用 k-means++ 改进。

> 注意：本项目对每个用户独立聚类，且 K 从 2 到 10 遍历。如果用 k-means++，每个 K 都要重新初始化，计算量稍大，但聚类质量更好。

3.3 项目实现步骤

1. 实现四种距离矩阵计算方法，对比性能；
2. 实现随机初始化和 k-means++ 初始化；
3. 对比两种初始化的 SSE 和迭代次数。

3.4 完整代码（距离矩阵与初始化策略）

```
# ============================================================
# 课时3：距离矩阵与初始化策略
# ============================================================

import numpy as np
import time



# ------------------------------------------------------------
# 1. 四种距离矩阵计算方法
# ------------------------------------------------------------
print("=" * 60)
print("1. 四种距离矩阵计算方法对比")
print("=" * 60)

np.random.seed(42)
n, d, k = 10000, 2, 5
X = np.random.randn(n, d)
centers = np.random.randn(k, d)

# 方法1：双重循环
t0 = time.time()
dist1 = np.zeros((n, k))
for i in range(n):
    for j in range(k):
        dist1[i, j] = np.sum((X[i] - centers[j]) ** 2)
t1 = time.time() - t0
print(f"双重循环：{t1:.4f} 秒")

# 方法2：单循环
t0 = time.time()
dist2 = np.zeros((n, k))
for j in range(k):
    dist2[:, j] = np.sum((X - centers[j]) ** 2, axis=1)
t2 = time.time() - t0
print(f"单循环：{t2:.4f} 秒（加速 {t1/t2:.1f} 倍）")
```

```
# 方法3：广播
t0 = time.time()
diff = X[:, None, :] - centers[None, :, :]
dist3 = (diff ** 2).sum(axis=2)
t3 = time.time() - t0
print(f"广播: {t3:.6f} 秒（加速 {t1/t3:.0f} 倍）")

# 方法4：矩阵展开
t0 = time.time()
X_sq = np.sum(X ** 2, axis=1, keepdims=True)
C_sq = np.sum(centers ** 2, axis=1, keepdims=True).T
cross = X @ centers.T
dist4 = X_sq + C_sq - 2 * cross
t4 = time.time() - t0
print(f"矩阵展开: {t4:.6f} 秒（加速 {t1/t4:.0f} 倍）")

# 验证结果一致
print(f"\n结果一致:")
print(f"  方法1 vs 方法2: {np.allclose(dist1, dist2)}")
print(f"  方法1 vs 方法3: {np.allclose(dist1, dist3)}")
print(f"  方法1 vs 方法4: {np.allclose(dist1, dist4)}")

# ------------------------------------------------------------
# 2. 随机初始化
# ------------------------------------------------------------
print("\n" + "=" * 60)
print("2. 随机初始化")
print("=" * 60)

def random_init(X, k, random_state=None):
    """随机初始化：从样本中随机选择k个点。"""
    n = X.shape[0]
    rng = np.random.default_rng(random_state)
    indices = rng.choice(n, k, replace=False)
    return X[indices].copy()

centers_random = random_init(X, k=3, random_state=42)
```

```
print(f"随机初始化中心:\n{centers_random}")

# ------------------------------------------------------------
# 3. k-means++ 初始化
# ------------------------------------------------------------
print("\n" + "=" * 60)
print("3. k-means++ 初始化")
print("=" * 60)

def kmeans_plus_plus_init(X, k, random_state=None):
    """k-means++ 初始化：初始中心尽可能分散。"""
    n, d = X.shape
    rng = np.random.default_rng(random_state)
    centers = []

    # 第一个中心：随机选择
    first_idx = rng.integers(n)
    centers.append(X[first_idx])

    # 后续中心：按距离概率选择
    for _ in range(1, k):
        centers_arr = np.array(centers)
        # 计算每个点到已选中心的最短距离平方
        diff = X[:, None, :] - centers_arr[None, :, :]
        dist_sq = (diff ** 2).sum(axis=2)
        min_dist_sq = dist_sq.min(axis=1)

        # 按概率选择（距离越远概率越大）
        probs = min_dist_sq / min_dist_sq.sum()
        next_idx = rng.choice(n, p=probs)
        centers.append(X[next_idx])

    return np.array(centers)

centers_pp = kmeans_plus_plus_init(X, k=3, random_state=42)
print(f"k-means++ 初始化中心:\n{centers_pp}")

# 计算初始中心的分散程度（两两距离之和）
```

```
def center_spread(centers):
    """计算初始中心的分散程度（两两距离之和，越大越分散）。"""
    k = len(centers)
    total = 0
    for i in range(k):
        for j in range(i+1, k):
            total += np.linalg.norm(centers[i] - centers[j])
    return total

print(f"\n随机初始化分散程度: {center_spread(centers_random):.4f}")
print(f"k-means++ 分散程度: {center_spread(centers_pp):.4f}")
print(f"k-means++ 更分散: {center_spread(centers_pp) > center_spread(centers_random)}")

# ------------------------------------------------------------
# 4. 两种初始化的聚类效果对比
# ------------------------------------------------------------
print("\n" + "=" * 60)
print("4. 两种初始化的聚类效果对比")
print("=" * 60)

def kmeans(X, k, init="random", max_iter=300, tol=1e-4, random_state=None):
    """k-means（支持随机和k-means++初始化）。"""
    n, d = X.shape
    rng = np.random.default_rng(random_state)

    if init == "random":
        centers = random_init(X, k, random_state)
    else:
        centers = kmeans_plus_plus_init(X, k, random_state)

    for iteration in range(max_iter):
        diff = X[:, None, :] - centers[None, :, :]
        dist_sq = (diff ** 2).sum(axis=2)
        labels = dist_sq.argmin(axis=1)

        new_centers = np.zeros_like(centers)
        for i in range(k):
            mask = labels == i
```

```
            if mask.sum() > 0:
                new_centers[i] = X[mask].mean(axis=0)
            else:
                new_centers[i] = X[rng.integers(n)]

        center_shift = np.linalg.norm(new_centers - centers, axis=1).max()
        centers = new_centers
        if center_shift < tol:
            break

    diff = X[:, None, :] - centers[None, :, :]
    dist_sq = (diff ** 2).sum(axis=2)
    labels = dist_sq.argmin(axis=1)
    sse = dist_sq.min(axis=1).sum()
    return labels, centers, sse, iteration + 1


# 生成测试数据（3个簇）
np.random.seed(42)
X_test = np.vstack([
    np.random.randn(100, 2) + [0, 0],
    np.random.randn(100, 2) + [5, 5],
    np.random.randn(100, 2) + [0, 5]
])

# 多次运行对比
print(f"{'初始化':<15} {'SSE':<12} {'迭代次数':<10}")
print("-" * 40)
for seed in range(5):
    _, _, sse_r, n_iter_r = kmeans(X_test, k=3, init="random", random_state=seed)
    _, _, sse_p, n_iter_p = kmeans(X_test, k=3, init="kmeans++", random_state=seed)
    print(f"随机(seed={seed})   {sse_r:<12.4f} {n_iter_r:<10}")
    print(f"k-means++(seed={seed}) {sse_p:<12.4f} {n_iter_p:<10}")

print("\n结论：k-means++ 初始化通常 SSE 更低、迭代次数更少")
```

3.5 运行结果与解读

运行后展示：
- 四种距离矩阵计算方法的性能对比（广播和矩阵展开最快）；
- 随机初始化和 k-means++ 初始化的中心对比；
- k-means++ 的初始中心更分散；
- 多次运行对比两种初始化的 SSE 和迭代次数（k-means++ 通常更优）。

> 关键理解：
> 1. 距离矩阵用广播或矩阵展开实现，比循环快成百上千倍；
> 2. k-means++ 初始化让初始中心更分散，收敛更快，SSE 更低；
> 3. 本项目可以用 k-means++ 改进聚类质量。

3.6 当堂练习

1. 把数据量 n 改为 10 万，重新对比四种距离矩阵方法的性能；
2. 用 k-means++ 初始化替换手写 k-means 中的随机初始化，观察 SSE 变化；
3. 思考：为什么 k-means++ 的初始中心更分散？（提示：距离已选中心越远的点被选中概率越大）

3.7 课时小结

- 距离矩阵：广播实现（直观）或矩阵展开（最快）；
- 随机初始化：简单但可能选到相近点；
- k-means++ 初始化：初始中心分散，收敛快，SSE 低；
- k-means++ 算法：第一个随机，后续按距离概率选择；
- 本项目可用 k-means++ 改进聚类质量。

课时4：聚类评估指标（SSE、轮廓系数、邓恩指数）

4.1 课时目标

掌握三种聚类评估指标：SSE（簇内平方和）、轮廓系数（Silhouette Coefficient）、邓恩指数（Dunn Index），理解各自的优缺点。

4.2 知识点深度讲解

4.2.1 为什么需要聚类评估指标

k-means 需要指定 K（簇数），但最优 K 是多少？需要用评估指标来判断。

评估指标分类：
1. **内部指标**：只根据数据和聚类结果评估，不需要真实标签（本项目用这个）；
2. **外部指标**：需要真实标签，对比聚类结果与真实标签（本项目没有真实标签，不用）。

本项目用的三种内部指标：
1. **SSE**（簇内平方和）：衡量簇内紧密度；
2. **轮廓系数**（Silhouette Coefficient）：综合衡量簇内紧密度和簇间分离度；
3. **邓恩指数**（Dunn Index）：衡量簇间最小距离与簇内最大距离的比值。

4.2.2 SSE（簇内平方和）

公式：
$$
SSE = \sum_{i=1}^{k} \sum_{x \in C_i} |x - \mu_i|^2
$$

含义：所有点到各自簇中心的距离平方和。

特点：
- SSE 越小，簇内越紧密；
- SSE 随 K 增大单调下降（K 越大，每个簇越小，SSE 越小）；
- 不能直接用 SSE 选最优 K（K 越大 SSE 越小，会选 K=n）；
- 用"肘部法则"（Elbow Method）选最优 K：SSE 下降速度突然变缓的点。

肘部法则：
- 画出 SSE-K 曲线；
- 找到"肘部"（SSE 下降速度突然变缓的点）；
- 肘部对应的 K 就是最优 K。

4.2.3 轮廓系数（Silhouette Coefficient）

公式：
对于样本 i：
$$
s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}
$$

其中：
- $a(i)$：样本 i 到同簇其他点的平均距离（簇内不相似度）；

- $b(i)$：样本 i 到最近的其他簇的平均距离（簇间不相似度）。

取值范围：$[-1, 1]$
- 接近 1：样本聚类合理（簇内紧密，簇间分离）；
- 接近 0：样本在两个簇的边界；
- 接近 -1：样本聚类错误（应该分到其他簇）。

整体轮廓系数：所有样本轮廓系数的平均值。

特点：
- 综合衡量簇内紧密度和簇间分离度；
- 越接近 1 越好；
- 计算复杂度高（$O(n^2)$），大数据集慢；
- 凸形簇效果好，非凸形簇效果差。

4.2.4 邓恩指数（Dunn Index）

公式：
$$
Dunn = \frac{\min_{i \neq j} d(C_i, C_j)}{\max_{k} d'(C_k)}
$$

其中：
- 分子：不同簇之间的最小距离（簇间最小距离）；
- 分母：簇内最大距离（簇的直径）。

含义：簇间最小距离与簇内最大距离的比值。

特点：
- 邓恩指数越大，簇间越分离、簇内越紧密，聚类效果越好；
- 对异常值敏感（异常值会增大簇内最大距离，降低邓恩指数）；
- 计算复杂度高（需要计算所有点对距离）；
- 本项目用邓恩指数选最优 K。

本项目为什么用邓恩指数?
- 项目文档明确要求用邓恩指数；
- 邓恩指数综合衡量簇间分离和簇内紧密，比 SSE 更合理；
- 本项目数据量不大（每个用户最多几百个点），计算复杂度可接受。

4.2.5 三种指标对比

指标	衡量	越大越好?	计算复杂度	缺点
SSE	簇内紧密度	越小越好	O(nk)	随K增大单调下降
轮廓系数	簇内+簇间	越大越好	$O(n^2)$	非凸簇效果差
邓恩指数	簇间/簇内	越大越好	$O(n^2)$	对异常值敏感

4.3 项目实现步骤

1. 实现 SSE 计算;
2. 实现轮廓系数计算;
3. 实现邓恩指数计算;
4. 用模拟数据测试三种指标。

4.4 完整代码（聚类评估指标实现）

```
# ============================================================
# 课时4：聚类评估指标（SSE、轮廓系数、邓恩指数）
# ============================================================

import numpy as np


def haversine(lon1, lat1, lon2, lat2, unit="km"):
    """球面距离（向量化）。"""
    R = 6371.0
    lon1, lat1, lon2, lat2 = map(np.radians, [lon1, lat1, lon2, lat2])
    dlat = lat2 - lat1
    dlon = lon2 - lon1
    a = np.sin(dlat / 2) ** 2 + np.cos(lat1) * np.cos(lat2) * np.sin(dlon / 2) ** 2
    c = 2 * np.arcsin(np.sqrt(a))
    return R * c * (1000 if unit == "m" else 1)


def calc_sse(X, labels, centers):
    """计算 SSE（簇内平方和）。"""
    sse = 0
    for i in range(len(centers)):
        mask = labels == i
        if mask.sum() > 0:
            sse += np.sum((X[mask] - centers[i]) ** 2)
    return sse


def calc_silhouette(X, labels):
    """计算轮廓系数（Silhouette Coefficient）。

    对于每个点 i:
        a(i) = 到同簇其他点的平均距离
        b(i) = 到最近其他簇的平均距离
        s(i) = (b(i) - a(i)) / max(a(i), b(i))
    整体轮廓系数 = 所有点 s(i) 的平均值
```

```
    """
    n = len(X)
    unique_labels = np.unique(labels)
    k = len(unique_labels)

    # 计算距离矩阵（欧几里得距离）
    # X[:, None, :] - X[None, :, :] → (n, n, d)
    diff = X[:, None, :] - X[None, :, :]
    dist_matrix = np.sqrt((diff ** 2).sum(axis=2))   # (n, n)

    silhouettes = np.zeros(n)
    for i in range(n):
        own_cluster = labels[i]
        own_mask = labels == own_cluster
        own_count = own_mask.sum()

        if own_count <= 1:
            # 簇内只有自己，a(i)=0，s(i)=0
            silhouettes[i] = 0
            continue

        # a(i)：到同簇其他点的平均距离
        a_i = dist_matrix[i, own_mask].sum() / (own_count - 1)

        # b(i)：到最近其他簇的平均距离
        b_i = np.inf
        for j in unique_labels:
            if j == own_cluster:
                continue
            other_mask = labels == j
            if other_mask.sum() > 0:
                avg_dist = dist_matrix[i, other_mask].mean()
                b_i = min(b_i, avg_dist)

        if b_i == np.inf:
            silhouettes[i] = 0
        else:
            silhouettes[i] = (b_i - a_i) / max(a_i, b_i)
```

```
    return silhouettes.mean()


def calc_dunn_index(X, labels):
    """计算邓恩指数（Dunn Index）。

    Dunn = 簇间最小距离 / 簇内最大距离（簇直径）
    """
    unique_labels = np.unique(labels)
    k = len(unique_labels)

    # 计算距离矩阵
    diff = X[:, None, :] - X[None, :, :]
    dist_matrix = np.sqrt((diff ** 2).sum(axis=2))

    # 计算每个簇的直径（簇内最大距离）
    max_intra = 0
    for label in unique_labels:
        mask = labels == label
        if mask.sum() > 1:
            cluster_dists = dist_matrix[np.ix_(mask, mask)]
            max_intra = max(max_intra, cluster_dists.max())

    # 计算簇间最小距离（不同簇点对的最小距离）
    min_inter = np.inf
    for i in range(k):
        for j in range(i + 1, k):
            mask_i = labels == unique_labels[i]
            mask_j = labels == unique_labels[j]
            if mask_i.sum() > 0 and mask_j.sum() > 0:
                inter_dists = dist_matrix[np.ix_(mask_i, mask_j)]
                min_inter = min(min_inter, inter_dists.min())

    if max_intra == 0 or min_inter == np.inf:
        return 0

    return min_inter / max_intra
```

```
# ============================================================
# 测试三种指标
# ============================================================
if __name__ == "__main__":
    # 生成3个簇的模拟数据
    np.random.seed(42)
    X = np.vstack([
        np.random.randn(50, 2) + [0, 0],
        np.random.randn(50, 2) + [5, 5],
        np.random.randn(50, 2) + [0, 5]
    ])
    true_labels = np.array([0]*50 + [1]*50 + [2]*50)

    print("=" * 60)
    print("真实聚类（3个簇）的评估指标")
    print("=" * 60)
    true_centers = np.array([X[true_labels == i].mean(axis=0) for i in range(3)])
    sse = calc_sse(X, true_labels, true_centers)
    sil = calc_silhouette(X, true_labels)
    dunn = calc_dunn_index(X, true_labels)
    print(f"SSE: {sse:.4f}")
    print(f"轮廓系数: {sil:.4f}")
    print(f"邓恩指数: {dunn:.4f}")

    # 对比：随机聚类（效果差）
    print("\n" + "=" * 60)
    print("随机聚类的评估指标（对比）")
    print("=" * 60)
    random_labels = np.random.randint(0, 3, size=len(X))
    random_centers = np.array([X[random_labels == i].mean(axis=0) if (random_labels ==
i).sum() > 0 else np.zeros(2) for i in range(3)])
    sse_r = calc_sse(X, random_labels, random_centers)
    sil_r = calc_silhouette(X, random_labels)
    dunn_r = calc_dunn_index(X, random_labels)
    print(f"SSE: {sse_r:.4f}（比真实聚类大: {sse_r > sse}）")
    print(f"轮廓系数: {sil_r:.4f}（比真实聚类小: {sil_r < sil}）")
```

```
    print(f"邓恩指数: {dunn_r:.4f}（比真实聚类小: {dunn_r < dunn}）")

    # 不同 K 的指标变化
    print("\n" + "=" * 60)
    print("不同 K 的指标变化（用于选最优 K）")
    print("=" * 60)
    print(f"{'K':<5} {'SSE':<12} {'轮廓系数':<12} {'邓恩指数':<12}")
    print("-" * 45)

    def kmeans_simple(X, k, random_state=42):
        """简单 k-means（用于测试）。"""
        rng = np.random.default_rng(random_state)
        indices = rng.choice(len(X), k, replace=False)
        centers = X[indices].copy()
        for _ in range(100):
            diff = X[:, None, :] - centers[None, :, :]
            dist_sq = (diff ** 2).sum(axis=2)
            labels = dist_sq.argmin(axis=1)
            new_centers = np.array([X[labels == i].mean(axis=0) if (labels == i).sum()
> 0 else centers[i] for i in range(k)])
            if np.allclose(centers, new_centers):
                break
            centers = new_centers
        return labels, centers

    for k in range(2, 7):
        labels, centers = kmeans_simple(X, k)
        sse_k = calc_sse(X, labels, centers)
        sil_k = calc_silhouette(X, labels)
        dunn_k = calc_dunn_index(X, labels)
        print(f"{k:<5} {sse_k:<12.4f} {sil_k:<12.4f} {dunn_k:<12.4f}")

    print("\n结论：")
    print("  - SSE 随 K 增大单调下降（肘部法则选 K=3）")
    print("  - 轮廓系数在 K=3 时最大（真实簇数）")
    print("  - 邓恩指数在 K=3 时最大（真实簇数）")
```

4.5 运行结果与解读

运行后展示：
- 真实聚类（3个簇）的三种指标；
- 随机聚类的三种指标（对比，效果差）；
- 不同 K 的指标变化（K=2~6），用于选最优 K。

> 关键理解：
> 1. SSE 越小越好，但随 K 增大单调下降，用肘部法则选 K；
> 2. 轮廓系数越接近 1 越好，在真实 K 处最大；
> 3. 邓恩指数越大越好，在真实 K 处最大；
> 4. 本项目用邓恩指数选最优 K。

4.6 当堂练习

1. 把数据改为4个簇，重新运行，观察三种指标在 K=4 时是否最优；
2. 思考：为什么 SSE 不能直接选最优 K？（提示：K 越大 SSE 越小，会选 K=n）
3. 实现"肘部法则"的自动化检测（SSE 下降速度突然变缓的点）。

4.7 课时小结

- SSE：簇内平方和，越小越好，随 K 增大单调下降，用肘部法则；
- 轮廓系数：综合簇内紧密度和簇间分离度，越接近 1 越好；
- 邓恩指数：簇间最小距离/簇内最大距离，越大越好；
- 本项目用邓恩指数选最优 K；
- 三种指标在真实 K 处都表现最优。

课时5：最优K选择（K=2~10遍历）

5.1 课时目标

掌握最优K选择方法：对每个用户遍历 K=2~10，用邓恩指数选最优K，处理K上限问题。

5.2 知识点深度讲解

5.2.1 为什么需要选最优K

不同用户的驻留区域数量不同：
- 有人只有家和公司（2个驻留区域）；
- 有人有家、公司、商场（3个）；
- 有人有多个常去地点（4-5个）。

如果 K 固定为某个值，对某些用户可能不合适：
- K 太小：多个驻留区域被合并到一个簇；
- K 太大：一个驻留区域被拆成多个簇。

所以需要对每个用户独立选最优 K。

5.2.2 遍历 K=2~10

本项目对每个用户遍历 K=2 到 10：
1. 对每个 K 运行 k-means；
2. 计算邓恩指数；
3. 选邓恩指数最大的 K 作为最优 K；
4. 用最优 K 重新运行 k-means，得到最终聚类结果。

为什么 K 从 2 开始?
- K=1 没有意义（所有点在一个簇，无法区分驻留区域）；
- K=2 是最小的有意义的簇数。

为什么 K 到 10 结束?
- 用户一天内的驻留区域通常不超过 10 个；
- K 太大计算量增加，且容易过拟合；
- 10 是合理的上限。

5.2.3 K 上限问题（关键坑）

如果用户点数 n < K，k-means 无法初始化（需要选 K 个不重复的点），会报错：

```
ValueError: Cannot take a larger sample than population when 'replace=False'
```

解决方法：K 上限改为 `min(K_MAX, n)`，即 K 不能超过用户点数。

```
k_max = min(10, n)   # K 上限不超过用户点数
for k in range(2, k_max + 1):
    # 运行 k-means
    pass
```

> 这是实测踩过的坑！如果用户只有 3 个点，K=10 会报错。必须限制 K 上限。

5.2.4 多次运行取最优

k-means 对初始值敏感，每次运行可能得到不同结果。为了稳定，可以：
1. 对每个 K 运行多次（如 5 次）；
2. 取 SSE 最小的那次结果；
3. 再计算邓恩指数。

本项目简化处理：每个 K 运行 1 次（随机种子固定），结果可复现。

5.2.5 邓恩指数的经纬度距离

本项目的点是经纬度，计算距离时应该用 Haversine（球面距离），而不是欧几里得距离。

但邓恩指数的实现中，距离矩阵用的是欧几里得距离。对于小范围（城市内），经纬度的欧几里得距离与 Haversine 距离近似成正比（因为经纬度变化很小，球面近似平面），所以用欧几里得距离也可以。

> 严格来说应该用 Haversine，但城市范围内欧几里得距离足够准确，且计算更快。本项目用欧几里得距离。

5.3 项目实现步骤

1. 对每个用户遍历 K=2~10（K上限不超过点数）；
2. 对每个 K 运行 k-means，计算邓恩指数；
3. 选邓恩指数最大的 K；
4. 用最优 K 重新运行 k-means，得到最终聚类结果；
5. 输出每个用户的最优 K 和聚类结果。

5.4 完整代码（最优K选择）

```
# ============================================================
# 课时5：最优K选择（K=2~10遍历）
# ============================================================

import pandas as pd
import numpy as np
import os


def kmeans(X, k, max_iter=300, tol=1e-4, random_state=42):
    """手写 k-means。"""
    n, d = X.shape
    rng = np.random.default_rng(random_state)
    indices = rng.choice(n, k, replace=False)
    centers = X[indices].copy()

    for iteration in range(max_iter):
        diff = X[:, None, :] - centers[None, :, :]
        dist_sq = (diff ** 2).sum(axis=2)
        labels = dist_sq.argmin(axis=1)

        new_centers = np.zeros_like(centers)
        for i in range(k):
            mask = labels == i
            if mask.sum() > 0:
                new_centers[i] = X[mask].mean(axis=0)
            else:
                new_centers[i] = X[rng.integers(n)]

        center_shift = np.linalg.norm(new_centers - centers, axis=1).max()
        centers = new_centers
        if center_shift < tol:
            break

    diff = X[:, None, :] - centers[None, :, :]
    dist_sq = (diff ** 2).sum(axis=2)
```

```
    labels = dist_sq.argmin(axis=1)
    sse = dist_sq.min(axis=1).sum()
    return labels, centers, sse


def calc_dunn_index(X, labels):
    """计算邓恩指数。"""
    unique_labels = np.unique(labels)
    k = len(unique_labels)

    diff = X[:, None, :] - X[None, :, :]
    dist_matrix = np.sqrt((diff ** 2).sum(axis=2))

    # 簇内最大距离（簇直径）
    max_intra = 0
    for label in unique_labels:
        mask = labels == label
        if mask.sum() > 1:
            cluster_dists = dist_matrix[np.ix_(mask, mask)]
            max_intra = max(max_intra, cluster_dists.max())

    # 簇间最小距离
    min_inter = np.inf
    for i in range(k):
        for j in range(i + 1, k):
            mask_i = labels == unique_labels[i]
            mask_j = labels == unique_labels[j]
            if mask_i.sum() > 0 and mask_j.sum() > 0:
                inter_dists = dist_matrix[np.ix_(mask_i, mask_j)]
                min_inter = min(min_inter, inter_dists.min())

    if max_intra == 0 or min_inter == np.inf:
        return 0
    return min_inter / max_intra


def select_best_k(X, k_min=2, k_max=10, random_state=42):
    """
```

```
    遍历 K=k_min~k_max，用邓恩指数选最优 K。

    Args:
        X: (n, d) 样本矩阵
        k_min: 最小 K
        k_max: 最大 K
        random_state: 随机种子

    Returns:
        best_k: 最优 K
        best_labels: 最优 K 对应的簇标签
        best_centers: 最优 K 对应的簇中心
        best_dunn: 最优邓恩指数
        k_results: 每个 K 的结果列表 [(k, dunn, sse, labels, centers), ...]
    """
    n = X.shape[0]

    # 关键：K 上限不能超过点数
    actual_k_max = min(k_max, n)
    if actual_k_max < k_min:
        # 点数太少，无法聚类，所有点在一个簇
        labels = np.zeros(n, dtype=int)
        centers = X.mean(axis=0, keepdims=True)
        return 1, labels, centers, 0, []

    k_results = []
    best_k = k_min
    best_dunn = -1
    best_labels = None
    best_centers = None

    for k in range(k_min, actual_k_max + 1):
        labels, centers, sse = kmeans(X, k, random_state=random_state)
        dunn = calc_dunn_index(X, labels)
        k_results.append((k, dunn, sse, labels, centers))

        if dunn > best_dunn:
            best_dunn = dunn
```

```
            best_k = k
            best_labels = labels
            best_centers = centers

    return best_k, best_labels, best_centers, best_dunn, k_results


# ============================================================
# 测试最优K选择
# ============================================================
if __name__ == "__main__":
    RESULT_DIR = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\交通时空大数据分析系统-14天授课笔记_第2版\result"

    # 读取阶段1完整数据
    full_path = os.path.join(RESULT_DIR, "stage1_full.csv")
    if not os.path.exists(full_path):
        print("请先运行 tr_stage1.py")
        exit()

    df = pd.read_csv(full_path, dtype={"imsi": str, "time": str,
                                       "lac_id": str, "cell_id": str, "laci": str})
    print(f"数据: {len(df)} 行, {df['imsi'].nunique()} 用户")

    # 选一个点数较多的用户测试
    user_counts = df.groupby("imsi").size().sort_values(ascending=False)
    test_imsi = user_counts.index[0]
    user_df = df[df["imsi"] == test_imsi].copy()
    print(f"\n测试用户: {test_imsi}")
    print(f"  点数: {len(user_df)}")

    # 排除阶段1出行点（阶段2只对非出行点聚类）
    non_travel = user_df[user_df["is_travel_stage1"] == False].copy()
    print(f"  非出行点数: {len(non_travel)}")

    if len(non_travel) < 2:
        print("  非出行点太少，无法聚类")
    else:
```

```
        X = non_travel[["lon", "lat"]].values
        best_k, best_labels, best_centers, best_dunn, k_results = select_best_k(X)

        print(f"\n最优 K: {best_k}")
        print(f"最优邓恩指数: {best_dunn:.4f}")
        print(f"各簇点数: {[np.sum(best_labels == i) for i in range(best_k)]}")

        print(f"\n各 K 的邓恩指数:")
        print(f"{'K':<5} {'邓恩指数':<12} {'SSE':<15} {'各簇点数'}")
        print("-" * 55)
        for k, dunn, sse, labels, centers in k_results:
            cluster_sizes = [np.sum(labels == i) for i in range(k)]
            print(f"{k:<5} {dunn:<12.4f} {sse:<15.6f} {cluster_sizes}")

    # 测试 K 上限问题
    print("\n" + "=" * 60)
    print("测试 K 上限问题（点数少的用户）")
    print("=" * 60)
    small_users = user_counts[user_counts < 5].index
    if len(small_users) > 0:
        small_imsi = small_users[0]
        small_df = df[df["imsi"] == small_imsi]
        print(f"用户 {small_imsi}: {len(small_df)} 点")
        X_small = small_df[["lon", "lat"]].values
        best_k, _, _, _, _ = select_best_k(X_small)
        print(f"  最优 K: {best_k}（K 上限自动限制为 {min(10, len(small_df))}）")
```

5.5 运行结果与解读

运行后展示：
- 测试用户的最优 K 和邓恩指数；
- 各 K 的邓恩指数、SSE、各簇点数；
- K 上限问题的测试（点数少的用户自动限制 K 上限）。

> 关键理解：
> 1. 最优 K 由邓恩指数最大决定；
> 2. K 上限不能超过用户点数（否则报错）；
> 3. 不同用户的最优 K 不同（有人2个驻留区域，有人3-4个）。

5.6 当堂练习

1. 选3个不同用户，分别运行最优K选择，对比最优K是否相同；
2. 思考：为什么邓恩指数最大的 K 是最优 K？（提示：邓恩指数大意味着簇间分离、簇内紧密）
3. 尝试用轮廓系数代替邓恩指数选最优 K，对比结果差异。

5.7 课时小结

- 最优K选择：遍历 K=2~10，邓恩指数最大的 K 为最优；
- K 上限：不能超过用户点数（关键坑）；
- 不同用户最优 K 不同；
- 用最优 K 重新运行 k-means 得到最终聚类结果。

课时6：单点成簇判定

6.1 课时目标

理解单点成簇的含义，掌握单点成簇判定方法，将单点成簇的点标记为出行点。

6.2 知识点深度讲解

6.2.1 什么是单点成簇

单点成簇（Singleton Cluster）指聚类后某个簇只有 1 个点。

为什么会出现单点成簇?

1. **异常点**：某个点远离其他所有点，k-means 把它单独聚成一个簇；
2. **出行点**：用户在移动过程中经过的点，不在任何驻留区域，被单独聚成一个簇；
3. **K 太大**：K 超过真实驻留区域数，多余的簇可能只有 1 个点。

本项目中的处理：单点成簇的点标记为出行点（不像是驻留区域）。

6.2.2 为什么单点成簇=出行点

业务逻辑：
- 驻留区域应该有多个点（用户在该区域停留一段时间，产生多条信令）；
- 如果一个簇只有 1 个点，说明用户在该位置只出现过一次，不像是驻留；
- 更可能是用户出行过程中经过的点（短暂连接某个基站）。

所以单点成簇的点标记为出行点，进入阶段3的出行点集合。

6.2.3 单点成簇判定实现

```
# 统计每个簇的点数
cluster_sizes = pd.Series(labels).value_counts()

# 找出单点成簇的簇标签
singleton_clusters = cluster_sizes[cluster_sizes == 1].index

# 单点成簇的点标记为出行点
is_singleton = np.isin(labels, singleton_clusters)
```

6.2.4 单点成簇的比例分析

统计单点成簇的点数占比，可以判断聚类质量：
- 单点成簇比例高：K 太大或数据噪声大；
- 单点成簇比例低：聚类质量好。

本项目中，单点成簇的点通常占 5%-15%。

6.2.5 单点成簇与阶段1出行点的关系

- 阶段1出行点：速度>8km/h 的点（明显在移动）；
- 阶段2出行点（单点成簇）：速度不高但只出现一次的点（可能在移动或短暂停留）；
- 两者合并后进入阶段3的出行点集合。

> 阶段2的作用是"补充"阶段1漏判的出行点：有些点速度不高（可能在等红灯、慢速行驶），但只出现一次，不像是驻留，阶段2把这些点也标记为出行。

6.3 项目实现步骤

1. 对每个用户聚类后，统计每个簇的点数；
2. 找出单点成簇的簇；
3. 将单点成簇的点标记为出行点；
4. 合并阶段1和阶段2的出行点；
5. 统计单点成簇的比例。

6.4 完整代码（单点成簇判定）

```
# ============================================================
# 课时6：单点成簇判定
# ============================================================

import pandas as pd
import numpy as np
import os



def kmeans(X, k, max_iter=300, tol=1e-4, random_state=42):
    """手写 k-means。"""
    n, d = X.shape
    rng = np.random.default_rng(random_state)
    indices = rng.choice(n, k, replace=False)
    centers = X[indices].copy()
    for iteration in range(max_iter):
        diff = X[:, None, :] - centers[None, :, :]
        dist_sq = (diff ** 2).sum(axis=2)
        labels = dist_sq.argmin(axis=1)
        new_centers = np.zeros_like(centers)
        for i in range(k):
            mask = labels == i
            if mask.sum() > 0:
                new_centers[i] = X[mask].mean(axis=0)
            else:
                new_centers[i] = X[rng.integers(n)]
        center_shift = np.linalg.norm(new_centers - centers, axis=1).max()
        centers = new_centers
        if center_shift < tol:
            break
    diff = X[:, None, :] - centers[None, :, :]
    dist_sq = (diff ** 2).sum(axis=2)
    labels = dist_sq.argmin(axis=1)
    sse = dist_sq.min(axis=1).sum()
    return labels, centers, sse
```

```
def calc_dunn_index(X, labels):
    """计算邓恩指数。"""
    unique_labels = np.unique(labels)
    k = len(unique_labels)
    diff = X[:, None, :] - X[None, :, :]
    dist_matrix = np.sqrt((diff ** 2).sum(axis=2))
    max_intra = 0
    for label in unique_labels:
        mask = labels == label
        if mask.sum() > 1:
            max_intra = max(max_intra, dist_matrix[np.ix_(mask, mask)].max())
    min_inter = np.inf
    for i in range(k):
        for j in range(i + 1, k):
            mask_i = labels == unique_labels[i]
            mask_j = labels == unique_labels[j]
            if mask_i.sum() > 0 and mask_j.sum() > 0:
                min_inter = min(min_inter, dist_matrix[np.ix_(mask_i, mask_j)].min())
    if max_intra == 0 or min_inter == np.inf:
        return 0
    return min_inter / max_intra



def select_best_k(X, k_min=2, k_max=10, random_state=42):
    """遍历 K=2~10，邓恩指数选最优 K。"""
    n = X.shape[0]
    actual_k_max = min(k_max, n)
    if actual_k_max < k_min:
        labels = np.zeros(n, dtype=int)
        centers = X.mean(axis=0, keepdims=True)
        return 1, labels, centers, 0, []
    best_k = k_min
    best_dunn = -1
    best_labels = None
    best_centers = None
    for k in range(k_min, actual_k_max + 1):
        labels, centers, sse = kmeans(X, k, random_state=random_state)
```

```
        dunn = calc_dunn_index(X, labels)
        if dunn > best_dunn:
            best_dunn = dunn
            best_k = k
            best_labels = labels
            best_centers = centers
    return best_k, best_labels, best_centers, best_dunn, []


# ============================================================
# 主流程：对所有用户聚类，判定单点成簇
# ============================================================
if __name__ == "__main__":
    RESULT_DIR = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\交通时空大数据分析系统-14天授课笔记_第2版\result"

    full_path = os.path.join(RESULT_DIR, "stage1_full.csv")
    df = pd.read_csv(full_path, dtype={"imsi": str, "time": str,
                                       "lac_id": str, "cell_id": str, "laci": str})
    print(f"数据: {len(df)} 行, {df['imsi'].nunique()} 用户")

    # 初始化阶段2标签
    df["cluster_id"] = -1          # 簇ID（-1表示未聚类）
    df["best_k"] = 0               # 最优K
    df["is_singleton"] = False     # 是否单点成簇
    df["is_travel_stage2"] = False # 阶段2出行点（单点成簇）

    total_singleton = 0
    total_clustered = 0
    user_best_k = []

    # 对每个用户聚类
    for imsi, user_df in df.groupby("imsi"):
        # 排除阶段1出行点（阶段2只对非出行点聚类）
        non_travel = user_df[user_df["is_travel_stage1"] == False]
        if len(non_travel) < 2:
            continue
```

```
        X = non_travel[["lon", "lat"]].values
        best_k, labels, centers, best_dunn, _ = select_best_k(X)

        # 记录簇标签
        df.loc[non_travel.index, "cluster_id"] = labels
        df.loc[non_travel.index, "best_k"] = best_k
        total_clustered += len(non_travel)
        user_best_k.append(best_k)

        # 判定单点成簇
        cluster_sizes = pd.Series(labels).value_counts()
        singleton_clusters = cluster_sizes[cluster_sizes == 1].index
        if len(singleton_clusters) > 0:
            is_singleton = np.isin(labels, singleton_clusters)
            df.loc[non_travel.index[is_singleton], "is_singleton"] = True
            df.loc[non_travel.index[is_singleton], "is_travel_stage2"] = True
            total_singleton += is_singleton.sum()

# 合并阶段1和阶段2出行点
df["is_travel_stage12"] = df["is_travel_stage1"] | df["is_travel_stage2"]

# 统计
print(f"\n{'='*60}")
print("阶段2聚类统计")
print(f"{'='*60}")
print(f"  聚类用户数: {len(user_best_k)}")
print(f"  聚类点数: {total_clustered}")
print(f"  最优K分布:")
for k in range(1, 11):
    count = user_best_k.count(k)
    if count > 0:
        print(f"    K={k}: {count} 用户")
print(f"  单点成簇点数: {total_singleton}")
print(f"  单点成簇占比: {total_singleton/total_clustered*100:.1f}%")
print(f"\n出行点统计:")
print(f"  阶段1出行点: {df['is_travel_stage1'].sum()}")
print(f"  阶段2出行点(单点成簇): {df['is_travel_stage2'].sum()}")
print(f"  合并出行点: {df['is_travel_stage12'].sum()}")
```

```
    print(f"  驻留点: {(~df['is_travel_stage12']).sum()}")

    # 保存
    output_path = os.path.join(RESULT_DIR, "stage2_cluster.csv")
    df[["imsi", "time", "lac_id", "cell_id", "lon", "lat",
        "is_travel_stage1", "cluster_id", "best_k",
        "is_singleton", "is_travel_stage2", "is_travel_stage12"]
    ].to_csv(output_path, index=False, encoding="utf-8")
    print(f"\n保存: {output_path}")
```

6.5 运行结果与解读

运行后展示:
- 聚类用户数、聚类点数;
- 最优K分布（K=2最多，K=3次之，...）;
- 单点成簇点数和占比;
- 阶段1、阶段2、合并出行点统计;
- 保存 stage2_cluster.csv。

> 关键理解:
> 1. 最优K分布: K=2最多（大部分用户只有家和公司两个驻留区域）;
> 2. 单点成簇占比通常 5%-15%;
> 3. 阶段2补充了阶段1漏判的出行点;
> 4. 合并后出行点进入阶段3。

6.6 当堂练习

1. 统计单点成簇的点的速度分布，与阶段1出行点对比;
2. 思考: 为什么单点成簇的点可能是出行点? （提示: 驻留区域应该有多个点）
3. 尝试把单点成簇阈值改为2（簇内点数<=2都算出行），对比结果差异。

6.7 课时小结

- 单点成簇: 聚类后某个簇只有1个点;
- 单点成簇=出行点: 驻留区域应该有多个点，单点不像是驻留;
- 阶段2补充阶段1漏判的出行点;
- 合并阶段1和阶段2出行点进入阶段3;
- 最优K分布: K=2最多。

课时7：阶段2完整实现（tr_stage2.py）

7.1 课时目标

将阶段2所有逻辑整合为完整模块 tr_stage2.py。

7.2 知识点深度讲解

7.2.1 模块结构

tr_stage2.py 的结构：
1. 配置区；
2. 工具函数（kmeans、calc_dunn_index、select_best_k）；
3. 主函数 stage2()；
4. 入口。

7.2.2 完整流程

```
读取 stage1_full.csv
  ↓
对每个用户：
  排除阶段1出行点
  遍历 K=2~10（K上限不超过点数）
  邓恩指数选最优K
  用最优K运行k-means
  判定单点成簇→标记为阶段2出行点
  ↓
合并阶段1+阶段2出行点
  ↓
输出 stage2_cluster.csv + stage2_travel_points.csv
```

7.3 项目实现步骤

按上述流程编写完整代码。

7.4 完整代码（tr_stage2.py）

```
# ============================================================
# 阶段2模块：tr_stage2.py
# 功能：k-means聚类、最优K选择、单点成簇判定
# 运行：python tr_stage2.py
# 输入：stage1_full.csv
# 输出：stage2_cluster.csv, stage2_travel_points.csv
# ============================================================

import os
import pandas as pd
import numpy as np

RESULT_DIR = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\交通时空大数据分析系统-14天授课笔记_第2版\result"
os.makedirs(RESULT_DIR, exist_ok=True)

K_MIN = 2
K_MAX = 10
RANDOM_STATE = 42


def kmeans(X, k, max_iter=300, tol=1e-4, random_state=RANDOM_STATE):
    """手写 k-means。"""
    n, d = X.shape
    rng = np.random.default_rng(random_state)
    indices = rng.choice(n, k, replace=False)
    centers = X[indices].copy()
    for iteration in range(max_iter):
        diff = X[:, None, :] - centers[None, :, :]
        dist_sq = (diff ** 2).sum(axis=2)
        labels = dist_sq.argmin(axis=1)
        new_centers = np.zeros_like(centers)
        for i in range(k):
            mask = labels == i
            if mask.sum() > 0:
                new_centers[i] = X[mask].mean(axis=0)
```

```
            else:
                new_centers[i] = X[rng.integers(n)]
        center_shift = np.linalg.norm(new_centers - centers, axis=1).max()
        centers = new_centers
        if center_shift < tol:
            break
    diff = X[:, None, :] - centers[None, :, :]
    dist_sq = (diff ** 2).sum(axis=2)
    labels = dist_sq.argmin(axis=1)
    sse = dist_sq.min(axis=1).sum()
    return labels, centers, sse


def calc_dunn_index(X, labels):
    """计算邓恩指数。"""
    unique_labels = np.unique(labels)
    k = len(unique_labels)
    diff = X[:, None, :] - X[None, :, :]
    dist_matrix = np.sqrt((diff ** 2).sum(axis=2))
    max_intra = 0
    for label in unique_labels:
        mask = labels == label
        if mask.sum() > 1:
            max_intra = max(max_intra, dist_matrix[np.ix_(mask, mask)].max())
    min_inter = np.inf
    for i in range(k):
        for j in range(i + 1, k):
            mask_i = labels == unique_labels[i]
            mask_j = labels == unique_labels[j]
            if mask_i.sum() > 0 and mask_j.sum() > 0:
                min_inter = min(min_inter, dist_matrix[np.ix_(mask_i, mask_j)].min())
    if max_intra == 0 or min_inter == np.inf:
        return 0
    return min_inter / max_intra


def select_best_k(X, k_min=K_MIN, k_max=K_MAX, random_state=RANDOM_STATE):
    """遍历 K，邓恩指数选最优 K。"""
```

```
    n = X.shape[0]
    actual_k_max = min(k_max, n)
    if actual_k_max < k_min:
        labels = np.zeros(n, dtype=int)
        centers = X.mean(axis=0, keepdims=True)
        return 1, labels, centers, 0.0
    best_k = k_min
    best_dunn = -1.0
    best_labels = None
    best_centers = None
    for k in range(k_min, actual_k_max + 1):
        labels, centers, sse = kmeans(X, k, random_state=random_state)
        dunn = calc_dunn_index(X, labels)
        if dunn > best_dunn:
            best_dunn = dunn
            best_k = k
            best_labels = labels
            best_centers = centers
    return best_k, best_labels, best_centers, best_dunn


def stage2():
    """执行阶段2完整流程。"""
    print("=" * 60)
    print("阶段2：k-means聚类与最优K选择")
    print("=" * 60)

    # 读取阶段1数据
    full_path = os.path.join(RESULT_DIR, "stage1_full.csv")
    if not os.path.exists(full_path):
        print("错误：请先运行 tr_stage1.py")
        return None
    df = pd.read_csv(full_path, dtype={"imsi": str, "time": str,
                                       "lac_id": str, "cell_id": str, "laci": str})
    print(f"数据：{len(df)} 行，{df['imsi'].nunique()} 用户")

    # 初始化阶段2标签
    df["cluster_id"] = -1
```

```
df["best_k"] = 0
df["is_singleton"] = False
df["is_travel_stage2"] = False

total_singleton = 0
total_clustered = 0
user_best_k = []
processed_users = 0

# 对每个用户聚类
for imsi, user_df in df.groupby("imsi"):
    non_travel = user_df[user_df["is_travel_stage1"] == False]
    if len(non_travel) < 2:
        continue

    X = non_travel[["lon", "lat"]].values
    best_k, labels, centers, best_dunn = select_best_k(X)

    df.loc[non_travel.index, "cluster_id"] = labels
    df.loc[non_travel.index, "best_k"] = best_k
    total_clustered += len(non_travel)
    user_best_k.append(best_k)
    processed_users += 1

    # 单点成簇判定
    cluster_sizes = pd.Series(labels).value_counts()
    singleton_clusters = cluster_sizes[cluster_sizes == 1].index
    if len(singleton_clusters) > 0:
        is_singleton = np.isin(labels, singleton_clusters)
        df.loc[non_travel.index[is_singleton], "is_singleton"] = True
        df.loc[non_travel.index[is_singleton], "is_travel_stage2"] = True
        total_singleton += is_singleton.sum()

# 合并出行点
df["is_travel_stage12"] = df["is_travel_stage1"] | df["is_travel_stage2"]

# 统计
print(f"\n聚类统计:")
```

```
    print(f"  处理用户数: {processed_users}")
    print(f"  聚类点数: {total_clustered}")
    print(f"  最优K分布:")
    for k in range(1, K_MAX + 1):
        count = user_best_k.count(k)
        if count > 0:
            print(f"    K={k}: {count} 用户 ({count/processed_users*100:.1f}%)")
    print(f"  单点成簇点数: {total_singleton}
({total_singleton/total_clustered*100:.1f}%)")

    print(f"\n出行点统计:")
    print(f"  阶段1出行点: {df['is_travel_stage1'].sum()}")
    print(f"  阶段2出行点(单点成簇): {df['is_travel_stage2'].sum()}")
    print(f"  合并出行点: {df['is_travel_stage12'].sum()}")
    print(f"  驻留点: {(~df['is_travel_stage12']).sum()}")

    # 输出
    cluster_path = os.path.join(RESULT_DIR, "stage2_cluster.csv")
    df[["imsi", "time", "lac_id", "cell_id", "lon", "lat",
        "is_travel_stage1", "cluster_id", "best_k",
        "is_singleton", "is_travel_stage2", "is_travel_stage12"]
    ].to_csv(cluster_path, index=False, encoding="utf-8")
    print(f"\n聚类结果: {cluster_path}")

    travel_path = os.path.join(RESULT_DIR, "stage2_travel_points.csv")
    df[df["is_travel_stage12"]][
        ["imsi", "time", "lac_id", "cell_id", "lon", "lat"]
    ].to_csv(travel_path, index=False, encoding="utf-8")
    print(f"合并出行点: {travel_path}")

    print("\n" + "=" * 60)
    print("阶段2完成！")
    print("=" * 60)
    return df


if __name__ == "__main__":
    stage2()
```

7.5 运行结果与解读

运行后展示每步统计，输出两个文件。

7.6 当堂练习

1. 运行 tr_stage2.py，记录最优K分布和单点成簇比例；
2. 检查 stage2_cluster.csv 的前20行，理解各字段含义。

7.7 课时小结

- tr_stage2.py 整合阶段2所有逻辑；
- 对每个用户独立聚类，邓恩指数选最优K；
- 单点成簇标记为出行点；
- 合并阶段1+阶段2出行点。

课时8：阶段2结果分析与可视化

8.1 课时目标

分析阶段2结果，可视化聚类结果、最优K分布、单点成簇分布。

8.2 知识点深度讲解

分析维度：
1. 最优K分布（柱状图）；
2. 聚类结果可视化（选一个用户，画散点图按簇着色）；
3. 单点成簇空间分布；
4. 阶段1 vs 阶段2出行点对比。

8.3 完整代码（阶段2结果分析与可视化）

```
# ============================================================
# 课时8：阶段2结果分析与可视化
# ============================================================

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import os

plt.rcParams["font.sans-serif"] = ["SimHei"]
plt.rcParams["axes.unicode_minus"] = False

RESULT_DIR = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\交通时空大数据分析系统-14天授课笔记_第2版\result"
FIG_DIR = os.path.join(RESULT_DIR, "figures")
os.makedirs(FIG_DIR, exist_ok=True)

df = pd.read_csv(os.path.join(RESULT_DIR, "stage2_cluster.csv"),
                 dtype={"imsi": str, "time": str, "lac_id": str, "cell_id": str})
print(f"数据：{len(df)} 行，{df['imsi'].nunique()} 用户")

# 图1：最优K分布
print("生成图1：最优K分布")
user_k = df[df["best_k"] > 0].groupby("imsi")["best_k"].first()
k_dist = user_k.value_counts().sort_index()

fig, ax = plt.subplots(figsize=(10, 5))
ax.bar(k_dist.index, k_dist.values, color="steelblue", edgecolor="black")
ax.set_xlabel("最优 K")
ax.set_ylabel("用户数")
ax.set_title(f"最优 K 分布（共{len(user_k)}用户）")
ax.set_xticks(k_dist.index)
for i, v in enumerate(k_dist.values):
    ax.text(k_dist.index[i], v + 0.5, str(v), ha="center")
plt.tight_layout()
plt.savefig(os.path.join(FIG_DIR, "fig_stage2_best_k.png"), dpi=150,
```

```
bbox_inches="tight")
plt.close()

# 图2：选一个用户的聚类结果
print("生成图2：用户聚类结果示例")
user_counts = df.groupby("imsi").size().sort_values(ascending=False)
test_imsi = user_counts.index[0]
user_df = df[df["imsi"] == test_imsi]
clustered = user_df[user_df["cluster_id"] >= 0]

fig, ax = plt.subplots(figsize=(10, 8))
# 阶段1出行点（灰色x）
travel1 = user_df[user_df["is_travel_stage1"] == True]
ax.scatter(travel1["lon"], travel1["lat"], c="gray", marker="x", s=30, label="阶段1出行
点", alpha=0.5)
# 聚类点（按簇着色）
if len(clustered) > 0:
    cmap = plt.cm.get_cmap("tab10", clustered["best_k"].iloc[0] if len(clustered) > 0
else 3)
    for cid in sorted(clustered["cluster_id"].unique()):
        cluster_pts = clustered[clustered["cluster_id"] == cid]
        center = cluster_pts[["lon", "lat"]].mean()
        ax.scatter(cluster_pts["lon"], cluster_pts["lat"], s=20, alpha=0.7,
                   label=f"簇{int(cid)}({len(cluster_pts)}点)")
        ax.scatter(center["lon"], center["lat"], c="black", marker="*", s=150)
# 单点成簇（红色圈）
singleton = clustered[clustered["is_singleton"] == True]
if len(singleton) > 0:
    ax.scatter(singleton["lon"], singleton["lat"], facecolors="none", edgecolors="red",
               s=100, linewidths=2, label=f"单点成簇({len(singleton)}点)")

ax.set_xlabel("经度")
ax.set_ylabel("纬度")
ax.set_title(f"用户 {test_imsi} 聚类结果（最优K={clustered['best_k'].iloc[0] if
len(clustered)>0 else 'N/A'}）")
ax.legend(bbox_to_anchor=(1.05, 1), loc="upper left")
plt.tight_layout()
plt.savefig(os.path.join(FIG_DIR, "fig_stage2_cluster_example.png"), dpi=150,
```

```
bbox_inches="tight")
plt.close()

# 图3：阶段1 vs 阶段2出行点对比
print("生成图3：阶段1 vs 阶段2出行点对比")
fig, axes = plt.subplots(1, 2, figsize=(14, 6))
# 空间分布
axes[0].scatter(df["lon"], df["lat"], c="lightgray", s=1, alpha=0.3, label="所有点")
axes[0].scatter(df[df["is_travel_stage1"]]["lon"], df[df["is_travel_stage1"]]["lat"],
                c="blue", s=5, alpha=0.5, label=f"阶段1出行
({df['is_travel_stage1'].sum()})")
axes[0].scatter(df[df["is_travel_stage2"]]["lon"], df[df["is_travel_stage2"]]["lat"],
                c="red", s=5, alpha=0.5, label=f"阶段2出行
({df['is_travel_stage2'].sum()})")
axes[0].set_xlabel("经度"); axes[0].set_ylabel("纬度")
axes[0].set_title("出行点空间分布"); axes[0].legend()
# 数量对比
stages = ["阶段1", "阶段2", "合并", "驻留"]
counts = [df["is_travel_stage1"].sum(), df["is_travel_stage2"].sum(),
          df["is_travel_stage12"].sum(), (~df["is_travel_stage12"]).sum()]
axes[1].bar(stages, counts, color=["blue", "red", "purple", "green"],
edgecolor="black")
axes[1].set_ylabel("点数"); axes[1].set_title("出行点 vs 驻留点数量")
for i, v in enumerate(counts):
    axes[1].text(i, v + 50, str(v), ha="center")
plt.tight_layout()
plt.savefig(os.path.join(FIG_DIR, "fig_stage2_travel_compare.png"), dpi=150,
bbox_inches="tight")
plt.close()

print(f"\n图片保存在：{FIG_DIR}")
print(f"  fig_stage2_best_k.png")
print(f"  fig_stage2_cluster_example.png")
print(f"  fig_stage2_travel_compare.png")
```

8.4 运行结果与解读

生成3张图：最优K分布、用户聚类结果示例、出行点对比。

8.5 当堂练习

1. 查看生成的3张图，分析最优K分布特征；
2. 选3个不同用户画聚类结果，对比驻留区域数量。

8.6 课时小结

- 最优K分布：K=2最多；
- 聚类结果：每个用户的驻留区域可视化；
- 单点成簇：补充阶段1漏判的出行点；
- 阶段2完成，进入阶段3规则精判。

三、本日小结

1. **k-means原理**：最小化SSE，分配-更新迭代，一定会收敛但可能局部最优；
2. **手写k-means**：广播距离矩阵、空簇处理、收敛检查；
3. **初始化策略**：随机 vs k-means++（后者初始中心更分散、收敛更快、SSE更低）；
4. **评估指标**：SSE（肘部法则）、轮廓系数（越接近1越好）、邓恩指数（越大越好）；
5. **最优K选择**：遍历K=2~10，邓恩指数选最优，K上限不超过点数（关键坑）；
6. **单点成簇**：簇内只有1个点→标记为出行点，补充阶段1漏判；
7. **tr_stage2.py**：完整实现阶段2，输出聚类结果和合并出行点；
8. **结果可视化**：最优K分布、用户聚类示例、出行点对比。

四、本日作业

1. 运行 tr_stage2.py，记录最优K分布和单点成簇比例；
2. 运行可视化代码，查看3张图，写一段分析；
3. 思考：为什么用邓恩指数而不是SSE选最优K？（提示：SSE随K增大单调下降）
4. 预习 Day06：思考"阶段3如何用规则精判每个点是驻留还是出行？"